

Researcher guide: `ops/` CLIs, batches, Modal, and `ennbo`

Yubo repository

1 Repository layout and Python path

Run CLIs from the repository root. Typical invocations use `./ops/....py` (shebang scripts); subprocesses assume this layout.

2 Modal (platform)

Modal (<https://modal.com>) runs containerized Python on cloud GPUs. This repo uses it in several ways:

- **Single UHD jobs:** `ops/exp_uhd.py` modal packages selected directories, optionally builds the sibling `enn` tree inside the image, and runs one experiment; logs stream to the local terminal.
- **UHD batch workers:** `ops/uhd_batch.py` modal and batch deploy or call a Modal app (`yubo_uhd_batch`) that runs `ops/exp_uhd.py` local per job; results land in Modal Dicts and are merged with `collect`.
- **Experiment batches:** `ops/modal_batches.py` wraps `modal deploy/run` for `experiments/modal_batches` with a `MODAL_TAG` environment variable so multiple logical deployments coexist.
- **Timing sweep:** `ops/single_run_time.py` targets `experiments/modal_timing_sweep.py`.
- **Synthetic sine benchmarks:** `ops/synthetic_sine_benchmark_batches.py` targets `experiments/modal_s`.

Install the Modal Python SDK (`pip install modal`), authenticate per Modal docs, and ensure the modal CLI is on your PATH.

3 The `ennbo` library and local `enn` code

`ennbo` is the PyPI-installable name used in `admin/install.sh` (`pip install ennbo --no-deps`) for epistemic nearest-neighbors (ENN) surrogate functionality used in optimizers and analysis (e.g. `enn.enn.enn_class`, `enn_fit`, benchmarks in `analysis/fitting_time/`).

The Modal image for full UHD runs (`ops/modal_uhd_runner_impl.py`) also copies a sibling check-out `../enn` into the container and builds the Rust extension with `maturin`, then installs it editable. For heavy ENN/Turbo work, keep local `enn` and Python `ennbo` installs aligned with project pins.

4 UHD configuration TOML and ops/exp_uhd.py

Ultra-high-dimensional (UHD) experiments are driven by a TOML file with an `[uhd]` table. Keys are validated and parsed in `ops/exp_uhd_parse.py` (required vs optional keys, optimizer-specific blocks, ENN/Bayesian exploration sections).

Local single run.

```
./ops/exp_uhd.py local path/to/config.toml
```

Modal single run.

```
./ops/exp_uhd.py modal path/to/config.toml \  
  [--log-file run.log] [--gpu A100]
```

GPU names follow Modal's device strings (e.g. T4, A10, A100, H100).

`ops/exp_uhd_full.py` is a variant that uses eager imports instead of the lazy `common.im` pattern; behavior is otherwise the same for `local/modal`.

5 ops/uhd_batch.py: replicated UHD runs (local or Modal)

This tool repeats the same UHD configuration for multiple random seeds, writing traces under `results/uhd/` by default (hashed per config; see `ops/uhd_batch_core.py`).

Local (threads + subprocesses).

```
./ops/uhd_batch.py local config.toml --num-reps N \  
  [--workers W] [--results-dir DIR]
```

Each replication runs `ops/exp_uhd.py local` in a subprocess; stdout is parsed for `EVAL:` lines to build `traces/*.jsonl`.

Modal (one config, many reps). Deploy the app once (see Modal dashboard or project docs), then:

```
./ops/uhd_batch.py modal config.toml --num-reps N \  
  [--results-dir DIR]
```

Use `collect`, `status`, and `cleanup` subcommands to pull results from Modal Dicts (`uhd_batch_results`, `uhd_batch_submitted`) and reset state when needed.

Modal (many configs from a prep function).

```
./ops/uhd_batch.py batch module.path.prep_fn \  
  [--results-dir DIR]
```

The prep callable must accept `results_dir` and return a **list of pairs** (`cfg_dict`, `num_reps`) where `cfg_dict` holds `[uhd]` fields suitable for `exp_uhd`. Example: `experiments.uhd_batch_preps.prep_uhd_batch`. This is *not* the same return type as `experiments.batch_preps.prep_*` (see below).

6 prep_* batches: two pipelines

6.1 General experiments: ops/batches.py

```
./ops/batches.py BATCH_TAG [--max-parallel P] \  
  [--dry-run] [--results-dir results]
```

BATCH_TAG names a function in `experiments.batch_preps`: either `prep_foo` or `foo` (the code prepends `prep_` if missing). Each `prep_*(results_dir)` returns a list of `ExperimentConfig` objects. The runner normalizes them to dicts and, unless `--dry-run`, spawns local processes that call `experiments.experiment_sampler.sampler` with `scan_local`. This is the main **local multiprocessing** workflow for paper-style batches.

6.2 UHD Modal batch preps

For `uhd_batch.py batch`, add new builders alongside `experiments/uhd_batch_preps.py`: return `list[tuple[dict, int]]`.

6.3 Timing sweep on Modal: ops/single_run_time.py

- `deploy` — deploy `experiments/modal_timing_sweep.py`.
- `submit BATCH_TAG --prep experiments.batch_preps.prep_...` — loads prep, requires each `ExperimentConfig` to have `num_reps == 1`, submits to Modal.
- `progress --prep ...` — counts completed local trace files for the same prep output (no Modal).
- `collect / stop` — fetch results or stop the app and run cleanup.

7 ops/modal_batches.py

Thin wrapper around modal CLI for `experiments/modal_batches_impl.py`: `deploy TAG`, `submit TAG BATCH_TAG`, `submit-force`, `collect TAG`, `status TAG`, `stop TAG`, `clean-up TAG`. Sets `MODAL_TAG` for isolated app names (`yubo_$TAG`).

8 ops/synthetic_sine_benchmark_batches.py

`Deploy/submit/collect/stop` for synthetic surrogate benchmarking on Modal, plus helpers to write and aggregate JSON artifacts. Uses per-tag Modal Dict names and `MODAL_TAG`; see script `--help`.

9 ops/catalog.py

Read-only introspection: `designers`, `policies`, `environments`, `uhd` (optimizer names for `optimizer=` in UHD TOML). Use to discover valid tags before editing configs.

10 ops/data.py

Inspect and manage UHD batch result trees under a `results-dir`: `ls`, `cat` (config and trace JSONL), `rm` (delete experiment subdirs by hash).

11 ops/experiment.py

Forwards all arguments to `experiments.experiment` Click CLI (broader experiment utilities). Run with `./ops/experiment.py ....`

12 ops/fit_mnist.py

Small MNIST training demo with timeout; options `--epochs`, `--batch-size`, `--lr`, `--timeout`. Uses `problems/mnist_classifier.py`.

13 ops/about_env.py

Legacy two-argument script: `python ops/about_env.py ENV_TAG POLICY_TAG` prints approximate noise level, mean absolute return, and parameter count for quick env sanity checks.

14 Summary table

Script	Role
<code>exp_uhd.py</code> / <code>exp_uhd_full.py</code>	Single UHD run from TOML (local / Modal)
<code>uhd_batch.py</code>	Many UHD reps; Modal <code>batch</code> from <code>(dict, reps)</code> preps
<code>batches.py</code>	Local pool of <code>ExperimentConfig</code> from <code>prep_*</code>
<code>single_run_time.py</code>	Modal timing sweep + prep progress
<code>modal_batches.py</code>	Modal wrapper for <code>modal_batches_impl</code>
<code>synthetic_sine_benchmark_batches.py</code>	Modal synthetic sine surrogate batches
<code>catalog.py</code>	List designers, policies, envs, UHD optimizers
<code>data.py</code>	List/cat/rm UHD batch results on disk
<code>experiment.py</code>	Forward to <code>experiments.experiment</code> CLI
<code>fit_mnist.py</code>	MNIST smoke / timing
<code>about_env.py</code>	Quick env/policy stats